

Simulazione del comportamento di stormi

Autori: Diego Mattioli e Matteo Procucci

Ultima modifica al progetto: 20/08/2025

Indirizzo della directory git: [git@github.com:DiegoMattioli/Progetto-Mattioli_Procucci.git](https://github.com/DiegoMattioli/Progetto-Mattioli_Procucci.git)

Per simulare il comportamento di alcuni boids in un cielo di dimensioni di 200 nelle ascisse e 200 nelle ordinate è stato creato una classe Boid, una classe Flock e una struct Parameters.

- La classe Boid crea una struttura dati caratterizzata da quattro double, due dei quali rappresentano le coordinate della posizione del Boid, un'ascissa e un'ordinata, e gli altri due le componenti della sua velocità, una v_x e una v_y .

Inoltre per ogni Boid è definito un vettore che contiene gli altri Boid che soddisfano il criterio di vicinanza con esso e le sue componenti della velocità di separazione, coesione e allineamento sono inizializzate a 0. Nella parte pubblica la classe contiene i metodi che permettono di sommare le ascisse, le ordinate, le componenti della velocità del Boid e delle tre velocità sopra citate, in aggiunta a ciò anche funzioni che aiutano con le formule di determinazione delle velocità di volo, come sommatorie e prodotti.

- La struct Parameters è una struttura dati definita da 5 double che rappresentano i 5 parametri di volo della simulazione (distanza, distanza di separazione, s , a , c).
- La classe Flock crea una struttura dati caratterizzata da un vettore che contiene tutti i Boid che fanno parte dello stormo e da un oggetto di tipo Parameters, che definisce le caratteristiche di volo del suddetto stormo.

La classe, nella sua parte pubblica, contiene i metodi che permettono di aggiungere Boids allo stormo, sia con coordinate e velocità iniziali specifiche che randomicamente generate, inoltre permette di ottenere informazioni sia sull' i -esimo Boid dello stormo, come ad esempio posizione, velocità e Boid ad esso vicini, che informazioni generali su velocità media e la sua deviazione standard che caratterizzano lo stormo. Infine consente di aggiornare contemporaneamente le posizioni e le velocità di tutti i Boid dello stormo.

Le due funzioni cardine del programma sono `update_position` e `update_velocity`, che permettono il proseguimento nel tempo della simulazione di volo:

- `Update_velocity` è il metodo che aggiorna la velocità secondo le velocità di separazione, coesione e di allineamento. Per fare ciò si trovano tramite un ciclo, per ogni Boid, i Boid a loro vicini e il centro di massa dello stormo e applica le regole di calcolo delle tre velocità, tramite funzioni che fanno le operazioni richieste, e le salva nelle tre variabili delle velocità inizializzate a 0, così da non cambiare le velocità dei Boid uno alla volta.

- `Update_position` è il metodo che somma le velocità, quella precedente e quelle calcolate in `update_velocity`, poi si calcola la nuova posizione come somma tra la posizione precedente e il prodotto tra la nuova velocità totale e il refresh rate.

Il codice prende in input i parametri di volo e il numero `n` di Boids che si vogliono generare randomicamente. Vengono creati due stormi, uno con i parametri messi in input e uno con dei parametri predefiniti ed una finestra grafica nella quale vengono rappresentati i Boid che mostrano l'applicazione delle regole di volo nella simulazione e il loro risultato. Inoltre fornisce in output la velocità media e la sua deviazione standard degli stormi.

Il codice è diviso in 6 traslation unit:

- `mattioli_procucci0.hpp`: contiene la definizione delle classi `Flock` e `Boid` e le dichiarazioni dei metodi delle due funzioni
- `mattioli_procucci0.cpp`: contiene la definizione dei metodi delle classi `Flock` e `Boid`
- `mattioli_procucci0-test`: contiene i Test Case, nei quali vengono testate i metodi delle classi e le eccezioni
- `mattioli_procucci-grafica.hpp`: contiene la dichiarazione di due funzioni utili per la parte grafica dalla libreria `sfml`
- `mattioli_procucci-grafica.cpp`: Contiene la creazione di uno sprite di dimensioni uguali a quelle dello spazio in cui si trovano i Boid, ad esso viene applicata una texture dove ogni pixel che corrisponde alle coordinate della posizione di un Boid viene colorato, lo sprite viene aggiornato ad ogni frame in modo da simulare lo spostamento dello stormo nel tempo.
- `main.cpp`: contiene un'applicazione delle strutture dati, dei metodi e dell'interfaccia grafica.

Per il corretto funzionamento del programma è necessario installare la libreria `sfml`, è possibile farlo tramite il terminale con il comando:

```
$ sudo apt-get install libsFML-dev
```

Per compilare ed eseguire il programma è necessario utilizzare i seguenti comandi nella directory contenente i file `mattioli_procucci0.cpp`, `mattioli_procucci-grafica.cpp` e i rispettivi header, `mattioli_procucci-test.cpp`, e `main.cpp`:

```
& cmake -S . -B build -G"Ninja multi-config"
```

```
$ cmake --build build --config Release
```

```
$ ./build/Release/progetto
```

I test sono stati sviluppati tramite `Doctest` e sono divisi in due Testcase, nel primo ci sono diversi Subcase nei quali viene testato singolarmente il corretto funzionamento dei metodi `add`, `update position` e `update_velocity`, quest'ultimo tra l'altro con due, tre e cinque Boid che compongono lo stormo.

Nel secondo Testcase, invece, viene fatto un test generale per controllare che i Boid non uscissero dall'area di simulazione.

I test sono stati pensati secondo la logica del problema e ogni valore nei Check è stato calcolato a mano.

esempio input/output:

Insert the number of Boids: 20

Select the value of the parameter d (distance between the boids): 90.

Select the value of the parameter ds (distance of separation between the boids): 10.

Select the value of the parameter 'a' (alignement of the flock, must be between 0 and 1): 0.4

Select the value of the parameter 'c' (coesione of the flock, must be between 0 and 1): 0.4

Select the value of the parameter 's' (separation of the flock, must be between 0 and 1): 0.4

closing window

Mean velocity: 0.596706 ± 1.14934

si consiglia di impostare un valore di d pari a circa 80./120. ed un valore di ds pari a circa 9./15.

La finestra grafica mostra il confronto fra lo stormo creato con i valori presi in input da tastiera (di colore Magenta) ed uno stormo creato ad hoc come esempio di stormo correttamente simulato (di colore Ciano)